

La Cave du Coin — Présentation technique du projet

Application de caisse et de gestion de stock pour un commerce de boissons, développée en solo, de la conception à l'hébergement en production réelle. Document préparé comme support d'entretien technique (stack, architecture, tests, infrastructure).

1. Contexte et objectif

La Cave du Coin est un commerce de boissons (Conakry, Guinée) qui avait besoin d'un logiciel de caisse et de suivi de stock adapté à sa réalité : plusieurs modes de paiement locaux (mobile money, vente à crédit informelle), un besoin fort de traçabilité anti-fraude (personnel non permanent), et une contrainte de coût (hébergement bon marché, pas de licence logicielle). Le projet couvre l'ensemble du cycle : conception du schéma de données, API, interface web, tests automatisés, containerisation, déploiement, environnement de staging et sauvegardes.

2. Stack technique

Domaine	Choix	Rôle
Langage / runtime	Python 3.12	Backend complet
Framework web	FastAPI 0.115 (+ Uvicorn ASGI)	API REST + rendu des pages HTML (Jinja2)
ORM	SQLAlchemy 2.0 (style déclaratif Mapped[...])	Modélisation des tables, requêtes
Migrations	Alembic 1.14	Versionner le schéma de base sans perte de données
Validation / schémas	Pydantic v2 + pydantic-settings	Validation des entrées API, config via .env
Base de données	SQLite (dev/tests) -> PostgreSQL 16 (prod)	Changement via une seule variable DATABASE_URL, zéro changement de code
Auth	PyJWT + passlib/bcrypt	Jetons de session, hachage des mots de passe
PDF	reportlab + qrcode	Reçus, devis, étiquettes produits, ce document
Email	smtplib (service mail.py)	Envoi de reçus par email
Frontend	Jinja2 (templates serveur) + JS natif + Chart.js (CDN)	Pas de framework JS : simplicité, pas de build step
Tests	pytest + httpx (TestClient FastAPI)	161 tests, unitaires et intégration HTTP
Conteneurisation	Docker (python:3.12-slim) + Docker Compose	Packaging et orchestration reproductible
Reverse proxy / HTTPS	Caddy 2	HTTPS automatique (Let's Encrypt) sans configuration manuelle
Hébergement	VPS Hetzner (CX22, ~4,5 €/mois)	Production réelle
CI/CD	Aucune (volontairement assumé, voir section 9)	Tests et déploiement lancés manuellement

3. Architecture applicative : séparation en couches

Le code applicatif (app/) suit une séparation stricte en quatre couches, pensée pour que la logique métier ne dépende jamais du framework web — donc testable en pur Python :

- `models.py` — les entités ORM (User, Product, Sale/SaleItem, Quote/QuoteItem, CashSession, Customer, StockMovement, Return, Log...) et les énumérations métier (rôles, modes de paiement, statuts).
- `schemas.py` — les schémas Pydantic, avec une séparation nette entre schémas d'entrée (*Create*) et de sortie (*Out*, avec `from_attributes=True` pour sérialiser directement les objets ORM en JSON).
- `services/*.py` — toute la logique métier (créer une vente, valider un mouvement de stock, convertir un devis...). Chaque fonction reçoit une session SQLAlchemy en paramètre et lève des exceptions métier dédiées (ex. `InsufficientStockError`, `CashSessionClosedError`) plutôt que de manipuler des codes HTTP.
- `routers/*.py` — la couche FastAPI, volontairement fine : authentification, appel du service correspondant, traduction des exceptions métier en codes HTTP (422/403/404/409), et écriture systématique dans le journal d'audit (`logs_service.record(...)`).

Bénéfice concret : les règles métier (calcul de monnaie, seuils anti-fraude, limite de crédit...) sont testées directement en appelant les fonctions de `services/` avec une base SQLite en mémoire, sans jamais démarrer de serveur HTTP — tests rapides et ciblés.

4. Fonctionnalités métier principales

- **Caisse / ventes** : panier, scan code-barres, plusieurs modes de paiement (espèces, mobile money, crédit, avoir, paycard...), calcul automatique de la monnaie, sessions de caisse (fond initial, écart théorique/physique, blocage automatique si écart > seuil).
- **Stock** : mouvements (entrée, casse, don, ajustement) avec **double validation obligatoire** par une personne différente de celle qui saisit, conversions carton/pack/unité automatiques, import CSV en masse, alertes de rupture prévisionnelle. Un ajustement peut être négatif (manque constaté au comptage) — tous les autres mouvements restent strictement positifs.
- **Péréemption par lot (FEFO)** : chaque entrée de stock peut porter sa propre date de péréemption (plusieurs livraisons du même produit peuvent coexister avec des dates différentes), y compris via l'import CSV en masse. La quantité restante par lot est **dérivée**, jamais stockée, en répartissant la consommation totale sur les lots du plus proche de la péréemption au plus lointain — alertes automatiques sur le tableau de bord.
- **Fiche d'inventaire à l'aveugle** : PDF listant les produits actifs avec des colonnes vides à remplir à la main, **sans afficher le stock théorique** — pensé pour un comptage physique fiable par une tierce personne, sans biais.
- **Clients** : vente à crédit (avec limite de créances simultanées par client), avoir suite à un retour, règlements partiels avec allocation à la dette la plus ancienne.
- **Devis** : créer un devis sans avoir besoin d'une session de caisse ouverte, prix figé au moment de la création, conversion en vente réelle en un clic (réutilise toute la logique de vente : stock, paiement, anti-fraude), PDF dédié. Numéro court et non-ambigu (ex. DEV-A7K9M) scannable directement via le QR code imprimé ; nom et téléphone du client obligatoires à la création pour permettre de retrouver un devis par recherche si le client perd son numéro.
- **Tableau de bord "Aujourd'hui"** : indicateurs du jour (CA, ventes, devis, mouvements de stock, écarts de caisse) — tuiles avec accent coloré par métrique et animation de comptage à chaque rafraîchissement — plus un camembert des produits vendus et un flux d'activité chronologique, construits à partir du journal d'audit existant, rafraîchis automatiquement toutes les 15 secondes (voir section 8 sur ce choix).
- **Statistiques / prévisions** : top produits, ventilation par catégorie/caissier, prévision de chiffre d'affaires par régression linéaire simple, prévision de rupture de stock.
- **Documents PDF** : reçus et devis au format ticket thermique (80 mm) avec QR code, étiquettes produit, reçus de règlement de créance, fiche d'inventaire.

5. Règles anti-fraude (le fil rouge du projet)

Un point volontairement mis en avant à chaque évolution du projet : chaque nouvelle fonctionnalité est systématiquement passée au filtre "est-ce que ça ouvre une brèche anti-fraude ?" avant d'être considérée terminée (voir section 10 pour un exemple concret découvert sur la fonctionnalité devis).

Règle	Seuil / mécanisme
-------	-------------------

Annulation d'un ticket par un caissier	possible seulement dans les 5 min suivant la vente, motif obligatoire
Annulation par un manager/admin	possible à tout moment
Vente/devis de plusieurs articles identiques	confirmation manuelle au-delà de 5 unités du même produit
Double scan du même produit	alerte si moins de 2 secondes entre deux scans
Écart de caisse à la fermeture	session bloquée si écart > 10 000 GNF, déblocage par un manager avec commentaire obligatoire
Mouvement de stock hors vente	doit être validé par une personne différente de celle qui l'a saisi
Ajustement d'inventaire	validation réservée à un Admin/Super Admin
Vente à crédit non réglée	maximum 2 créances en cours par client, sinon vente refusée
Prix de vente d'un produit	ne peut pas descendre sous le prix d'achat - 5 %, sauf marqué "promo"
Réimpression d'un reçu/devis	automatiquement marquée DUPLICATA
Toute action	journalisée (qui, quoi, quand, IP) dans un journal d'audit jamais supprimé

6. Base de données et migrations

SQLite en développement/tests, PostgreSQL 16 en production — le changement de moteur ne touche aucune ligne de code applicatif, uniquement la variable d'environnement `DATABASE_URL` (configuration via `pydantic-settings`).

Toutes les évolutions de schéma passent par Alembic (jamais de `create_all()` en production). Point technique notable : SQLite ne supporte pas nativement `ALTER TABLE` pour toutes les opérations, donc les migrations qui modifient une table existante utilisent le mode `batch_alter_table` d'Alembic (recréation de table en coulisse), activé conditionnellement selon le dialecte (SQLite uniquement, pas nécessaire sur PostgreSQL). Les migrations qui créent des tables neuves (ex. `quotes/quote_items`) utilisent directement `op.create_table(...)`. Les migrations sont appliquées automatiquement au démarrage du conteneur (`alembic upgrade head && uvicorn ...` dans le Dockerfile).

7. Stratégie de tests — 161 tests

La suite mélange volontairement deux niveaux, souvent dans un même fichier :

- **Tests unitaires (couche service)** : appellent directement les fonctions de services/ avec une base SQLite en mémoire (fixture `db_session`), sans passer par HTTP — rapides, ciblés sur une règle métier précise.
- **Tests d'intégration HTTP** : passent par un vrai `TestClient` FastAPI (fixture `client`) avec une authentification JWT réelle obtenue via `/api/auth/login` (fixture `auth_headers`) — valident le comportement de bout en bout, y compris les codes HTTP retournés.

Domaine testé	Fichier(s)
Règles anti-fraude transverses (annulation, double scan)	<code>test_antifraude_rules.py</code>
Ventes, reçus PDF, emails de reçu	<code>test_sales_and_receipts.py</code>

Sessions de caisse (ouverture/fermeture/écarts/résolution)	test_cash_sessions.py
Retours, avoir, crédit client, créances	test_returns_and_credit.py, test_customer_credit.py
Devis (création, conversion, expiration, recherche client, anti-fraude)	test_quotes.py (16 tests)
Tableau de bord "Aujourd'hui"	test_today_dashboard.py (4 tests)
Fiche d'inventaire à l'aveugle	test_inventory_sheet.py (4 tests)
Stock : conversions, marges, double validation, ajustement négatif, lots FEFO	test_products_stock_conversions.py
Import CSV (produits, mouvements de stock)	test_product_import.py, test_movement_import.py
Fournisseurs, étiquettes/codes-barres produits	test_suppliers.py, test_product_labels.py
Authentification, rôles, gestion des mots de passe	test_auth_permissions.py, test_password_management.py
Super Admin (gestion des comptes Admin)	test_super_admin.py
Prévisions (régression linéaire, rupture de stock, alertes de péremption)	test_predictions.py
Sauvegardes (SQLite, rétention, copie offsite)	test_backup.py
Envoi d'emails (SMTP)	test_mail.py

Exécution : pytest (configuration minimale via `pytest.ini`, aucun marqueur personnalisé). Aucune CI ne les exécute automatiquement — voir section 9.

8. Hébergement, containerisation et mise en production

L'application est packagée dans une image Docker unique (python:3.12-slim) et orchestrée via Docker Compose, avec des fichiers **combinables** plutôt qu'un seul fichier monolithique — le même code sert trois contextes différents :

Fichier	Rôle
docker-compose.yml	Base commune : service db (Postgres 16) + app (build de l'image). Toujours utilisé.
docker-compose.vps.yml	Overlay production : ajoute Caddy (reverse proxy + HTTPS automatique via Let's Encrypt).
docker-compose.lan.yml	Overlay alternatif : expose juste le port 8000 sur le réseau local (PC boutique, sans HTTPS).
docker-compose.staging.yml	Overlay staging : ports liés uniquement à 127.0.0.1 (jamais exposés publiquement), ajoute pgAdmin.

Le choix se fait via la variable `COMPOSE_FILE` dans `.env` (`prod = docker-compose.yml:docker-compose.vps.yml`). Le staging tourne comme un **projet Compose séparé** (`-p staging`), avec sa propre base et ses propres volumes, isolé de la prod, mais partage le réseau Docker `caddy-net` avec elle pour que Caddy puisse router vers son pgAdmin.

Mise en production : VPS Hetzner (Ubuntu, ~4,5 €/mois), pare-feu UFW n'autorisant que SSH/80/443. Déploiement d'une mise à jour : `git pull && docker compose up -d --build` (les migrations Alembic s'appliquent automatiquement au démarrage du conteneur). Aucun nom de domaine acheté n'était nécessaire pour obtenir un certificat HTTPS valide (via sslip.io, qui encode l'IP dans un sous-domaine) avant l'achat éventuel d'un vrai domaine.

Choix délibéré sur le tableau de bord "Aujourd'hui" (section 4) : rafraîchissement par `setInterval` côté JS toutes les 15 secondes plutôt que des websockets. Le besoin réel (un commerce, quelques mouvements par minute au plus) ne justifiait pas la complexité d'infrastructure supplémentaire (connexions persistantes, gestion de la reconnexion...) — ce même pattern de rafraîchissement périodique existait déjà sur la page de suivi de caisse.

Problème réel rencontré et corrigé : les fichiers CSS/JS étaient référencés par un chemin statique, donc les navigateurs continuaient de servir une version en cache après chaque déploiement (jusqu'à un hard refresh manuel). Corrigé avec un **cache-busting** simple : une variable `static_version` (l'heure de démarrage du process) injectée en paramètre `?v=` sur chaque `<link>/<script>` via un global Jinja2 — change automatiquement à chaque redémarrage de conteneur, donc à chaque déploiement, sans action manuelle.

9. Environnement de staging et refresh automatique depuis la prod

Un environnement de **staging** tourne en parallèle de la prod sur le même VPS (même code, base et ports isolés). Un script (`scripts/refresh_staging_from_prod.sh`), planifié par cron le **1er de chaque mois à 2h du matin**, le resynchronise automatiquement :

- `git pull` puis redémarrage du stack staging avec la dernière version du code.
- Vérifie si la prod contient de vraies données (`COUNT(products)+COUNT(sales)`) : si oui, `pg_dump -Fc` de la vraie prod, restauré dans staging via `pg_restore --clean --if-exists`.
- Si la prod est encore vide (avant lancement commercial), restaure un jeu de données de démonstration à la place — le staging n'est jamais vide, même en phase de pré-lancement.

Un **pgAdmin** dédié (accessible en HTTPS via Caddy sur un sous-domaine séparé) permet d'explorer visuellement les deux bases — prod et staging — sans jamais avoir besoin de SSH ni de taper de SQL brut en console. Pour cela, la base de production a été jointe au réseau Docker partagé `caddy-net` sous un alias dédié (`proddb`, distinct du db de staging pour éviter toute collision de nom) — uniquement pour que pgAdmin puisse s'y connecter ; l'application de prod continue, elle, de joindre sa base normalement, sans changement de comportement.

Point de vigilance identifié et documenté : ce refresh mensuel copie de **vraies données clients** (noms, téléphones, adresses, soldes) dans staging. Jugé acceptable dans ce contexte précis (structure mono-proprétaire, pas de tiers y ayant accès), mais l'accès à l'environnement de staging doit rester soumis à la même rigueur que la production — bon exemple de compromis pragmatique documenté plutôt qu'ignoré.

10. Sauvegardes et sécurité

Sauvegardes : `scripts/backup.py`, exécuté toutes les 4h via cron. Dump PostgreSQL (`pg_dump` dans le conteneur) ou copie de fichier pour SQLite selon l'environnement. Rétention configurable (14 jours par défaut, purge automatique des sauvegardes plus anciennes), copie optionnelle vers un stockage hors-site (non bloquant si indisponible).

Sécurité : mots de passe hachés (bcrypt via passlib, jamais stockés en clair), sessions authentifiées par JWT à durée limitée, quatre rôles hiérarchiques (Caissier < Manager < Admin < Super Admin) contrôlés par une dépendance FastAPI unique (`require_role(*roles)`) réutilisée sur chaque endpoint sensible, séparation des tâches sur les mouvements de stock (voir section 5), et journal d'audit exhaustif pour toute action mutante — la même table de logs qui sert désormais de base au tableau de bord "Aujourd'hui".

11. Un exemple concret d'itération avec revue anti-fraude

Bon exemple à raconter en entretien pour illustrer une hygiène d'ingénierie, pas juste "livrer une fonctionnalité qui marche" : lors de l'ajout des devis, la règle anti-fraude "confirmation obligatoire au-delà de 5 articles identiques" (déjà appliquée aux ventes) avait été **oubliée** à la création d'un devis. Comme la conversion d'un devis en vente forçait cette confirmation à True par construction, un devis de 50 unités identiques pouvait se convertir en vente sans jamais déclencher le contrôle. Le trou a été identifié en repensant le flux réel d'usage ("un client repart chercher du cash, on met sa vente en attente"), corrigé (la même vérification s'applique désormais aussi à la création d'un devis), testé, puis documenté dans le guide de formation de l'équipe.

Second exemple, découvert en répondant à une question opérationnelle concrète ("comment valider un inventaire en cas de manque constaté ?") plutôt qu'en relisant le code à froid : le champ qty d'un mouvement de stock exigeait une valeur strictement positive pour **tous** les types de mouvement, y compris "Ajustement" — alors qu'un ajustement doit justement pouvoir être négatif pour enregistrer un manque (vol, casse non déclarée). Autrement dit : la règle métier documentée ("comparer au stock théorique et ajuster") n'était en réalité pas réalisable pour le cas le plus important, un manque. Corrigé en ne relâchant la contrainte que pour ce type précis, testé, puis documenté. Illustre qu'une user story formulée comme une vraie question d'usage (pas "ajoute un champ X") révèle des trous que la relecture de code seule aurait pu manquer.

12. Limites connues et axes d'amélioration

Assumés et identifiés soi-même — bon signal en entretien de savoir nommer les limites de son propre travail plutôt que de les laisser découvrir :

- **Pas de CI/CD** : tests et déploiement lancés manuellement (pytest en local avant chaque push, puis `git pull && docker compose up -d --build` sur le VPS). Une GitHub Action lançant la suite de tests sur chaque push serait la prochaine amélioration naturelle.
- **Documentation désynchronisée, leçon apprise** : le nombre de tests dans le README a traîné obsolète pendant un moment (39 alors que le code était déjà à 143) avant d'être remarqué. Depuis, chaque fonctionnalité ajoutée s'accompagne systématiquement d'une mise à jour du README/guide de formation dans le même lot de changements — discipline plutôt que rattrapage ponctuel.
- **Pas de vrai temps réel** : rafraîchissement par sondage (15s) plutôt que websockets, choix délibéré de simplicité pour l'échelle actuelle du projet (voir section 8).
- **Développé sans Docker en local** à l'origine (pas installé sur la machine de développement) : les fichiers Compose ont été écrits selon des patterns standards puis validés directement sur le vrai VPS, plutôt que testés en local au préalable.

13. Pitch en une minute

"J'ai conçu et développé seul, de bout en bout, un logiciel de caisse et de gestion de stock pour un commerce réel — API FastAPI/SQLAlchemy typée, 161 tests automatisés (unitaires et intégration), règles anti-fraude explicites dictées par un vrai besoin métier, containerisé avec Docker et déployé sur un VPS avec HTTPS automatique. J'ai aussi mis en place un environnement de staging synchronisé automatiquement depuis la prod chaque mois, des sauvegardes régulières avec rétention, et je documente moi-même les limites de mes choix techniques (pas de CI encore, pas de websockets) plutôt que de les présenter comme un produit fini sans défaut."